

Competitive Programming

Section 30: GCD, LCM, and Diophantine Equations

Competitive Programming Course

2026-08-14

Table of contents

1	Goals and problem signals	3
2	GCD: the invariant behind Euclid's algorithm	4
2.1	Trace: gcd(252, 105)	4
3	LCM: use the identity, but preserve the arithmetic order	5
3.1	Trace: a=84, b=126	5
4	Extended Euclid: construct Bezout coefficients	6
4.1	Trace: coefficients for 30 and 18	7
5	General modular inverse	8
5.1	Trace: inverse of 6 modulo 35	8
6	Linear Diophantine equations	9
6.1	Why this family contains every solution	9
6.2	Trace: solve $6x + 10y = 14$	10
7	Generalized Chinese Remainder Theorem	12
7.1	Turn the congruences into one equation	12
7.2	Construct one common value	12
7.3	Why the period is the lcm	13
7.4	Trace: merge $x \equiv 2 \pmod{6}$ and $x \equiv 5 \pmod{9}$	13
7.5	More than two congruences	14
8	Implementation notes	15
8.1	C++	15
8.2	Python	15
9	Failure modes and tests	16
10	Formative exercises	17
11	Exercises	18
12	Local workflow	19

1 Goals and problem signals

This section develops one chain of ideas:

```
Euclid computes gcd.  
Extended Euclid also constructs coefficients for the gcd.  
Those coefficients give modular inverses and solve  $ax + by = c$ .
```

The three local problems are deliberately close, but their outputs are different. Before coding, identify which question is being asked.

Statement signal	Mathematical object	First operation
“largest number dividing both”	$\gcd(a,b)$	Euclid
“smallest common multiple”	$\text{lcm}(a,b)$	divide by gcd before multiplying
“inverse modulo m ”	$a \cdot x + m \cdot y = 1$	extended Euclid, then normalize x
“find integers x,y ”	$a \cdot x + b \cdot y = c$	test $\gcd(a,b) \mid c$, then scale Bezout coefficients

The local inputs are positive integers. That keeps the implementation compact; for variants with negative inputs, first make the gcd nonnegative and take extra care with language-specific division and remainder rules.

2 GCD: the invariant behind Euclid's algorithm

For positive integers a and b , the greatest common divisor is the largest positive integer dividing both. Euclid's key identity is:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Write $a = q \cdot b + r$, where $r = a \bmod b$. A number divides both a and b exactly when it divides both b and $a - q \cdot b = r$. Thus a Euclid step preserves the *set* of common divisors, not merely its greatest member.

Pseudocode:

```
while b != 0:
    (a, b) = (b, a mod b)
return a
```

2.1 Trace: $\gcd(252, 105)$

Current a	Current b	Remainder $a \bmod b$	Next pair
252	105	42	(105, 42)
105	42	21	(42, 21)
42	21	0	(21, 0)

The algorithm returns 21. At every row, the common divisors are unchanged; when the second value is zero, every positive divisor of the first value is a common divisor, so that first value is the gcd.

Euclid runs in $O(\log \min(a, b))$ iterations. Consecutive Fibonacci numbers are the familiar near-worst case, so this bound remains excellent even for 10^{18} -scale values.

3 LCM: use the identity, but preserve the arithmetic order

For the positive inputs in problem A:

```
gcd(a,b) * lcm(a,b) = a*b
```

The identity follows prime by prime. If prime p occurs with exponents u and v in a and b , then gcd uses $\min(u,v)$ and lcm uses $\max(u,v)$. Their exponents add to $u+v$, exactly the exponent in $a*b$.

Compute:

```
g = gcd(a,b)
l = (a / g) * b
```

The parentheses are part of the algorithm. $a*b/g$ can overflow before the division even when the final lcm fits. Dividing first removes the common factor. It does **not** prove that the final product fits; use `__int128` when the constraints permit lcm above `long long`.

3.1 Trace: a=84, b=126

```
gcd(84,126) = 42
lcm(84,126) = (84 / 42) * 126 = 252
```

Check both directions: $252/84=3$ and $252/126=2$, so it is a common multiple; the gcd identity proves it is the least one.

4 Extended Euclid: construct Bezout coefficients

Ordinary Euclid returns only $g = \text{gcd}(a,b)$. Extended Euclid also returns integers x,y satisfying:

$$a*x + b*y = g$$

This is Bezout's identity. The recursive base case is clear:

$$\text{when } b = 0: \quad a*1 + 0*0 = a$$

Now consider $\text{extended_gcd}(a,b)$ with $b \neq 0$. Euclid replaces (a,b) by the smaller pair:

$$(b, r), \text{ where } r = a \bmod b$$

The recursive call is therefore $\text{extended_gcd}(b,r)$. Apply the function's return contract to **these new arguments**: it returns $g,x1,y1$ such that

$$b*x1 + r*y1 = g$$

Here, $x1$ multiplies the recursive call's first argument b , and $y1$ multiplies its second argument r . They are not yet the coefficients of the original pair (a,b) .

Euclid's invariant gives $\text{gcd}(b,r) = \text{gcd}(a,b)$, so g is already the correct gcd. We only need to rewrite the equation using a and b . From integer division,

$$\begin{aligned} a &= \text{floor}(a/b)*b + r \\ r &= a - \text{floor}(a/b)*b \end{aligned}$$

Substitute this expression for r into the recursive equation:

$$\begin{aligned} b*x1 + r*y1 & \\ &= b*x1 + (a - \text{floor}(a/b)*b)*y1 \\ &= a*y1 + b*(x1 - \text{floor}(a/b)*y1) \end{aligned}$$

The last line now has the form required by the parent call:

$$a*x + b*y = g$$

Compare the coefficient multiplying a and the coefficient multiplying b :

```
x = y1
y = x1 - floor(a/b)*y1
```

This transformation does not guess new coefficients. It takes the equation proved by the recursive call and rewrites the same equation in terms of the parent's arguments.

Pseudocode, including the exact return contract:

```
extended_gcd(a, b):
    if b == 0:
        return (a, 1, 0)
    (g, x1, y1) = extended_gcd(b, a mod b)
    return (g, y1, x1 - floor(a/b)*y1)
```

4.1 Trace: coefficients for 30 and 18

The parent call reduces (30,18) to (18,12) because $30 \bmod 18 = 12$. Suppose the recursive call has found:

$$18*1 + 12*(-1) = 6$$

These are coefficients for 18 and 12, not yet for 30 and 18. Replace the remainder 12 by $30 - 18$:

$$\begin{aligned} 6 &= 18 - 12 \\ &= 18 - (30 - 18) \\ &= 30*(-1) + 18*2 \end{aligned}$$

Thus the parent returns:

$$\begin{aligned} g &= 6, x = -1, y = 2 \\ 30*(-1) + 18*2 &= 6 \end{aligned}$$

There are many valid coefficient pairs. Extended Euclid is useful because it constructs one deterministically in logarithmic time.

5 General modular inverse

An inverse of a modulo m is a residue x with:

$$a*x \equiv 1 \pmod{m}$$

This congruence is equivalent to the existence of an integer y with:

$$a*x + m*y = 1$$

Bezout's identity says an integer linear combination of a and m can equal 1 exactly when $\gcd(a,m)=1$. Thus:

$$\text{inverse exists} \iff \gcd(a,m) = 1$$

When extended Euclid returns $a*x + m*y = 1$, the coefficient x is an inverse. Normalize it because judges normally require the smallest nonnegative representative:

```
x = x mod m
if x < 0: x += m      # needed in C++ after a negative remainder
```

The local inverse problem defines the answer for $m=1$ to be 0. Handle it before the ordinary inverse logic; modulo one has only the residue zero.

5.1 Trace: inverse of 6 modulo 35

Extended Euclid finds:

$$6*6 + 35*(-1) = 1$$

Hence 6 is the inverse. Verify the required congruence, not ordinary division:

$$6*6 = 36 \equiv 1 \pmod{35}$$

For 6 modulo 9, $\gcd(6,9)=3$. Every combination $6*x+9*y$ is divisible by three, so it can never equal one. Returning any number as an inverse is wrong.

6 Linear Diophantine equations

Consider:

$$a*x + b*y = c$$

Every left-hand side is a multiple of $g=\gcd(a,b)$, so a necessary condition is $g \mid c$. It is also sufficient. If extended Euclid gives:

$$a*u + b*v = g$$

and $c = g*scale$, multiply both sides by $scale$:

$$a*(u*scale) + b*(v*scale) = c$$

This yields one particular solution:

$$\begin{aligned} x_p &= u*scale \\ y_p &= v*scale \end{aligned}$$

The complete family is useful in harder variants with nonnegative or bounded variables:

$$\begin{aligned} x &= x_p + (b/g)*t \\ y &= y_p - (a/g)*t \end{aligned}$$

for any integer t . Substitution shows the additional terms cancel.

6.1 Why this family contains every solution

Substitution proves that every integer t produces a valid solution, but we also need the reverse direction: every valid solution must be produced by some integer t .

Let (x,y) be any solution and (x_p,y_p) the particular solution above. Both satisfy the same equation:

$$\begin{aligned} a*x + b*y &= c \\ a*x_p + b*y_p &= c \end{aligned}$$

Subtract the second equation from the first. With $dx = x-x_p$ and $dy = y-y_p$, we obtain:

$$a \cdot dx + b \cdot dy = 0$$

Divide by g and write $A=a/g$, $B=b/g$:

$$A \cdot dx = -B \cdot dy$$

By removing the gcd, A and B are coprime. The right side is divisible by B , so $A \cdot dx$ is divisible by B . Because A and B are coprime, this forces dx itself to be divisible by B . Therefore, for some integer t :

$$dx = B \cdot t = (b/g) \cdot t$$

Substituting this back into $A \cdot dx = -B \cdot dy$ gives:

$$dy = -A \cdot t = -(a/g) \cdot t$$

Hence every solution has exactly the form:

$$\begin{aligned} x &= x_p + (b/g) \cdot t \\ y &= y_p - (a/g) \cdot t \end{aligned}$$

There are no other possibilities because the difference between any solution and the particular solution is forced to have these two multiples.

The divisibility step above is Euclid's lemma: if $\gcd(A,B)=1$ and B divides $A \cdot z$, then B divides z . To see why, Bezout gives integers p, q with $p \cdot A + q \cdot B = 1$. Multiply by z :

$$z = p \cdot (A \cdot z) + q \cdot (B \cdot z)$$

Both terms on the right are divisible by B , so z is divisible by B as well.

6.2 Trace: solve $6x + 10y = 14$

Extended Euclid may return:

$$6 \cdot 2 + 10 \cdot (-1) = 2$$

Here $g=2$ divides 14, and $scale=7$:

$$\begin{aligned} x_p &= 2 \cdot 7 = 14 \\ y_p &= -1 \cdot 7 = -7 \end{aligned}$$

Verification is mandatory:

$$6 \cdot 14 + 10 \cdot (-7) = 84 - 70 = 14$$

Since $b/g=5$ and $a/g=3$, every solution is:

$$\begin{aligned}x &= 14 + 5*t \\ y &= -7 - 3*t\end{aligned}$$

For example, $t=-2$ gives $(x,y)=(4,-1)$, and $6*4 + 10*(-1) = 14$.

For $6x+10y=15$, the gcd is still two but does not divide fifteen, so no integer solution exists.

7 Generalized Chinese Remainder Theorem

Suppose one integer must satisfy two congruences:

```
x == r1 (mod m1)
x == r2 (mod m2)
```

The classical Chinese Remainder Theorem is often stated only for coprime moduli. Competitive-programming problems do not always give that convenience, so we will derive the general version from the Diophantine equation we already know how to solve.

7.1 Turn the congruences into one equation

The first congruence means that, for some integer k ,

```
x = r1 + m1*k.
```

For this value to satisfy the second congruence, there must be an integer t such that

```
r1 + m1*k = r2 + m2*t.
```

Rearrange:

```
m1*k - m2*t = r2-r1.
```

This is a linear Diophantine equation. Let $g = \gcd(m1, m2)$. From the previous section, it has a solution exactly when

```
g divides r2-r1.
```

This is the compatibility condition. When the moduli are not coprime, the two residues must agree modulo their gcd. Checking whether $m1$ divides the difference would be too strict; we are free to add multiples of both moduli.

7.2 Construct one common value

Extended Euclid gives coefficients p, q with

```
m1*p + m2*q = g.
```

When g divides $r2-r1$, multiply by $(r2-r1)/g$:

$$m_1 * (p*(r_2-r_1)/g) + m_2 * (q*(r_2-r_1)/g) = r_2-r_1.$$

Comparing this with $m_1*k - m_2*t = r_2-r_1$, one valid choice is

$$k = p*(r_2-r_1)/g.$$

Therefore one common value is

$$x_0 = r_1 + m_1*k.$$

It may be negative or unnecessarily large. That does not make the construction wrong; it means we still need to identify the period of all common solutions.

7.3 Why the period is the lcm

Let x and y both satisfy the two congruences. Then $x-y$ is divisible by m_1 and by m_2 . Therefore it is divisible by $\text{lcm}(m_1, m_2)$.

Conversely, adding $\text{lcm}(m_1, m_2)$ to a solution adds a multiple of each modulus, so both remainders stay unchanged. Thus all solutions are exactly

$$x = x_0 + \text{lcm}(m_1, m_2)*z$$

for integers z . There are no missing solutions because the difference of any two solutions was forced to be a multiple of the lcm.

The smallest nonnegative answer is consequently

$$x_0 \text{ modulo } \text{lcm}(m_1, m_2).$$

This result is the **generalized Chinese Remainder Theorem**, usually shortened to **generalized CRT**. If the moduli are coprime, $g=1$, compatibility is automatic, and the period is m_1*m_2 : this is the familiar classical form.

7.4 Trace: merge $x \equiv 2 \pmod{6}$ and $x \equiv 5 \pmod{9}$

Here $\text{gcd}(6, 9)=3$, and 3 divides $5-2=3$, so the congruences are compatible. One Bezout identity is

$$6*(-1) + 9*1 = 3.$$

The scale is $(5-2)/3=1$, so $k=-1$ is valid:

$$x_0 = 2 + 6*(-1) = -4.$$

The common period is $\text{lcm}(6, 9)=18$. Normalize:

```
-4 modulo 18 = 14.
```

Indeed, 14 leaves remainder 2 modulo 6 and remainder 5 modulo 9.

For $x \equiv 1 \pmod{4}$ and $x \equiv 2 \pmod{6}$, the gcd is two but the residue difference is one. No common value exists.

7.5 More than two congruences

After merging two compatible congruences, their solutions form one new residue class modulo their lcm. You can merge that class with a third congruence using the same procedure, then continue. At every merge, perform the gcd compatibility test before constructing the new residue.

8 Implementation notes

8.1 C++

`std::gcd` is available through `<numeric>` (and therefore through the common contest include `bits/stdc++.h`). Store answers and Bezout coefficients in `long long` for the local 10^9 constraints. In larger variants, the update:

```
y = x1 - (a / b) * y1;
```

and the final scaling can require `__int128` even when `a` and `b` fit in `long long`.

For CRT, calculate the period as $(m_1/g)*m_2$, dividing before multiplying. The final period fits signed 64-bit under the local constraints, but a raw product such as $m_1*p*(r_2-r_1)/g$ may not. Normalize the intermediate coefficient modulo m_2/g and use `__int128` for the multiplication.

8.2 Python

Use `math.gcd` when only the gcd is needed. Python integers do not overflow, but recursive extended Euclid still needs its base case and exact quotient formula. Python normalizes `x % m` into $[0, m-1]$ for positive `m`; C++ does not, which is why the explicit correction is shown above.

For CRT, Python's `%` directly normalizes a constructed solution into $[0, lcm-1]$. Still compute the lcm as `m1//g*m2` so the arithmetic mirrors the proof and translates safely to fixed-width languages.

9 Failure modes and tests

Mistake	Minimal counterexample	Correct reaction
compute $a*b/g$ first	large values with a small gcd	divide by gcd first
assume every a has an inverse	$a=6, m=9$	print -1 because gcd is 3
use Fermat under a	inverse modulo 35	use extended Euclid
composite modulus		
scale before divisibility test	$6x+10y=15$	report impossible
reject negative coefficients	$6x+10y=14$ gives $y=-7$	integers may be negative
forget modulus one	$a=25, m=1$	output 0 in the local contract
assume CRT moduli are coprime	$x=2 \bmod 6, x=5 \bmod 9$	test the gcd condition
require equal residues	$2 \bmod 6, 5 \bmod 9$	compare residues modulo the gcd
forget CRT normalization	construction gives $x_0=-4$	reduce modulo the lcm

For every Diophantine output, substitute the printed values back into the original equation. This catches swapped coefficients, missing scale factors, and sign errors immediately.

10 Formative exercises

1. Compute $\gcd(391, 299)$ by Euclid. What invariant remains true after every remainder step?

Hint: write $391 = 1 \cdot 299 + 92$ and continue.

Answer: the gcd is 23; every step preserves the set of common divisors.

2. Does 14 have an inverse modulo 35? Explain without trying candidate residues.

Hint: an inverse would make an integer combination equal to one.

Answer: no. $\gcd(14, 35) = 7$, so every combination is divisible by seven.

3. Find one pair solving $15x + 21y = 12$.

Hint: first find a Bezout representation of $\gcd(15, 21)$.

Answer: $15 \cdot 3 + 21 \cdot (-2) = 3$; scale by four to get $(x, y) = (12, -8)$.

4. Describe all solutions to $6x + 10y = 14$ from the worked trace.

Hint: use $g=2$ and add a multiple of $(b/g, -a/g)$.

Answer: $x=14+5t, y=-7-3t$ for every integer t .

5. Decide whether $x \equiv 4 \pmod{6}$ and $x \equiv 10 \pmod{15}$ are compatible, then give their common period.

Hint: compare the residue difference with $\gcd(6, 15)$.

Answer: yes, because three divides six. Their common period is $\text{lcm}(6, 15) = 30$.

11 Exercises

Complete the five formative exercises above before implementing local Problems A–E; their hints and answers are deliberately included for self-checking. Problem F is intentionally not described here: treat it as blind recognition practice after completing the direct CRT problem E.

12 Local workflow

Run the section checker after implementing each problem:

```
CP_TARGET=solution python3 sections/30_gcd_lcm_diophantine/check.py
```

Before submission, check these exact questions:

1. For lcm, did I divide before multiplying?
2. For an inverse, did I test `gcd(a,m) == 1` and normalize the coefficient?
3. For $ax+by=c$, did I test `c % gcd(a,b)` before scaling?
4. Did I verify $a*x+b*y$ on a small trace, including a negative coefficient?
5. Do my integer types cover intermediate products under the stated bounds?
6. When merging congruences, did I test gcd compatibility and normalize modulo the lcm?